

El modelo entidad-relación

Módulo: Fundamentos de bases de datos relacionales

|AE5: Elaborar un modelo de datos de acuerdo a los estándares de modelamiento para resolver un problema de baja complejidad.

Introducción

El Modelo Entidad-Relación (ER) es uno de los modelos más utilizados para el diseño de bases de datos, ya que, entre otras cosas, este modelo conceptual **facilita la representación gráfica de las entidades y sus relaciones** en un esquema fácilmente comprensible.

En un Modelo Entidad-Relación, **las entidades representan objetos** o conceptos del mundo real, mientras que **las relaciones describen las interacciones entre estas entidades**. Estas relaciones pueden ser de diferentes tipos, como "uno a uno," "uno a muchos," o "muchos a muchos," definiendo la naturaleza de la conexión entre ellas.

En esta lección podrás explorar los **fundamentos del Modelo Entidad-Relación** en bases de datos, los distintos tipos de relación entre entidades, así como los tipos de datos y restricciones aplicables al modelo. También examinaremos las reglas de transformación para convertir un modelo ER en uno relacional y las técnicas de normalización para garantizar la eficiencia y consistencia de la base de datos. Al entender estos conceptos, podrás diseñar y gestionar bases de datos sólidas y bien estructuradas, capaces de satisfacer las necesidades de organizaciones modernas en el manejo de su información.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Conocer el proceso de abstracción en bases de datos
- Conocer el modelo conceptual de Entidad-Relación
- Definir de atributos e identificadores únicos
- Analizar los distintos tipos de relación entre entidades
- Diferenciar las entidades débiles y fuertes en bases de datos
- Analizar las diferencias entre el modelo relacional y el modelo conceptual
- Asignar tipos de datos y restricciones al modelo
- Reconocer las reglas de transformación y de normalización de un modelo relacional

El modelo Entidad-Relación

El modelo Entidad-Relación (ER, por si siglas en ingles Entity-Relationship) es un modelo conceptual de datos utilizado en el diseño de bases de datos para representar las relaciones entre entidades (objetos o conceptos) de un sistema. Proporciona una representación gráfica del esquema de la base de datos y sirve de modelo para diseñar e implementar una base de datos relacional.

El modelo ER consta de tres componentes principales:

- a.  **Entidades:** Las entidades representan objetos o conceptos del mundo real, como una persona, un lugar, un evento o una cosa. En una base de datos, las entidades se representan mediante tablas. Cada entidad tiene atributos que describen sus propiedades. Por ejemplo, una entidad "Cliente" puede tener atributos como `customer_id`, nombre, correo electrónico y dirección.
- b.  **Relaciones:** Las relaciones representan las asociaciones entre entidades. Describen cómo se relacionan las entidades entre sí y la cardinalidad de la relación (por ejemplo, uno a uno, uno a muchos, muchos a muchos). Las relaciones se representan mediante líneas que conectan las entidades relacionadas en el diagrama ER. Por ejemplo, una entidad "Cliente" puede tener una relación con una entidad "Pedido", indicando que un cliente puede realizar uno o más pedidos.
- c.  **Atributos:** Los atributos son las propiedades o características de las entidades. Proporcionan información adicional sobre las entidades y se representan como columnas en una tabla. Por ejemplo, la entidad "Cliente" puede tener atributos como `customer_id`, nombre, correo electrónico y dirección.

El modelo ER ayuda a los diseñadores de bases de datos a **comprender la estructura de la base de datos y las relaciones entre entidades**. Proporciona una representación visual del esquema de la base de datos, que sirve de base para crear la base de datos real utilizando un sistema de gestión de bases de datos relacionales (RDBMS). El modelo ER suele utilizarse durante las fases iniciales del diseño de la base de datos para capturar los requisitos y las relaciones antes de traducirlos a un esquema relacional.

Proceso de abstracción en Bases de Datos

En el contexto de las bases de datos, el proceso de abstracción se refiere a la técnica de ocultar los detalles complejos de cómo se almacenan los datos y se accede a ellos, y presentar a los usuarios una visión simplificada y coherente de los datos. Permite a los usuarios y aplicaciones interactuar con la base de datos sin necesidad de comprender las complejidades técnicas subyacentes.

Suele haber tres niveles de abstracción en un sistema de gestión de bases de datos (SGBD):

1. Nivel externo o de vista:

Es el nivel más alto de abstracción y se refiere a la forma en que los usuarios individuales o las aplicaciones perciben los datos. Define las distintas vistas o interfaces de usuario de la base de datos. Cada usuario o aplicación puede tener su propia vista personalizada, mostrando sólo los datos relevantes y ocultando el resto. Por ejemplo, un departamento de ventas puede tener una vista diferente de la base de datos que el departamento de inventario.

2. Nivel conceptual o lógico:

El nivel conceptual representa la estructura lógica general de toda la base de datos sin entrar en detalles de implementación. Trata de la organización de los datos a nivel conceptual, incluidas las relaciones entre las distintas entidades y sus atributos. El esquema conceptual o modelo de datos, como el modelo Entidad-Relación (ER), se utiliza para definir este nivel de abstracción.

3. Nivel interno o físico:

El nivel interno se ocupa del almacenamiento físico real y de los detalles de implementación de la base de datos a nivel de hardware. Describe cómo se almacenan, indexan y acceden los datos en los dispositivos de almacenamiento subyacentes. Este nivel se ocupa de la optimización del rendimiento, la indexación y otros detalles del almacenamiento físico.

El propósito de estos niveles de abstracción es proporcionar independencia de los datos. Lo ideal es que los cambios realizados en un nivel no afecten a los demás niveles. Por ejemplo, si el almacenamiento físico cambia debido a actualizaciones de hardware, los niveles lógico y de vista no deberían verse

afectados, garantizando que las aplicaciones y los usuarios puedan seguir accediendo a los datos sin modificaciones.

Utilizando este proceso de abstracción, las bases de datos pueden gestionarse de forma más eficiente y permitir un mantenimiento y una evolución más sencillos a lo largo del tiempo, sin interrumpir las aplicaciones y los usuarios que dependen de los datos.

Definición de atributos e identificadores únicos

En las bases de datos, los atributos y los identificadores únicos desempeñan un papel esencial en la **definición y organización de los datos**.

Definamos cada uno de estos conceptos:

→ Atributos:

En el contexto de las bases de datos, un atributo se refiere a una característica o propiedad que describe una entidad concreta en una tabla de la base de datos. Representa un único dato dentro de un registro o fila. Cada fila de una tabla de base de datos contiene un conjunto de atributos, y cada atributo tiene un tipo de datos específico que define el tipo de datos que puede contener (por ejemplo, texto, número, fecha).

Por ejemplo, una tabla de base de datos llamada "Employees", que almacena información sobre los empleados de una empresa. Los atributos de esta tabla pueden ser "EmployeeID", "FirstName", "LastName", "JobTitle", "Department" y "Salary". Cada atributo corresponde a una información sobre un empleado y, juntos, forman un registro completo que representa a un empleado individual en la tabla.

Los atributos son los **bloques de construcción de la estructura de una base de datos**, y son cruciales para organizar y recuperar datos de manera eficiente.

→ Identificadores únicos:

Un identificador único, también conocido como clave primaria, es un atributo especial o una combinación de atributos dentro de una tabla de base de datos que identifica de forma única cada registro (fila) de esa tabla. La clave primaria

garantiza que no haya dos filas en la tabla que tengan la misma combinación de valores para los atributos identificados. Proporciona una forma única de referenciar y acceder a registros individuales dentro de la tabla.

La clave primaria es esencial por varias razones: 📌

- ★ **Unicidad**: Garantiza que cada fila de la tabla sea distinta e identificable.
- ★ **Indexación**: Permite una búsqueda y recuperación más rápida de los datos, ya que los valores de la clave primaria suelen estar indexados por el sistema de gestión de bases de datos (SGBD).
- ★ **Relaciones**: En las bases de datos relacionales, las claves primarias se utilizan para establecer relaciones entre tablas a través de claves externas, que hacen referencia a la clave primaria de otra tabla.

Por ejemplo, en la tabla "Employees" mencionada anteriormente, el atributo "EmployeeID" podría servir como identificador único o clave primaria. Esto significa que no puede haber dos empleados con el mismo EmployeeID, lo que permite un acceso fácil e inequívoco a los registros individuales de los empleados.

Los atributos describen las características de los datos dentro de una base de datos, mientras que los identificadores únicos (claves primarias) garantizan que cada registro sea identificable de forma única y permiten una recuperación de datos y una gestión de relaciones eficaces.

Tipos de relaciones entre tablas en una Base de Datos relacional

En una base de datos relacional, las entidades se representan como tablas, y las relaciones entre entidades describen cómo se relacionan las tablas entre sí. Existen tres tipos principales de relaciones entre entidades en una base de datos:

Relación uno a uno (One-to-One):

En una relación uno a uno, cada registro de una tabla está asociado a un único registro de otra tabla, y viceversa. Este tipo de relación es relativamente poco frecuente en las bases de datos relacionales y suele utilizarse cuando dos entidades tienen una conexión única y singular.

- **Ejemplo:** Dos tablas, "Empleado" y "EmpleadoContacto". Cada empleado de la tabla "Empleado" puede tener un único registro correspondiente en la tabla "EmpleadoContacto", y viceversa, que representa la información de contacto de ese empleado concreto.

Relación uno a muchos (One-to-Many):

En una relación de uno a muchos, un registro de una tabla puede estar relacionado con varios registros de otra tabla, pero cada registro de la segunda tabla sólo puede estar asociado con un registro de la primera tabla. Es el tipo de relación más habitual en las bases de datos relacionales.

- **Ejemplo:** Dos tablas, "Departamento" y "Empleado". Cada departamento de la tabla "Departamento" puede tener varios empleados, pero cada empleado sólo puede pertenecer a un departamento. Por lo tanto, la tabla "Departamento" puede tener un registro relacionado con muchos registros de la tabla "Empleado".

Relación Muchos-Muchos (Many-to-Many):

En una relación muchos a muchos, cada registro de una tabla puede estar asociado a varios registros de otra tabla, y viceversa. Para representar este tipo de relación en una base de datos relacional, se utiliza una tabla intermedia, a menudo denominada tabla de unión o asociativa.

- **Ejemplo:** Dos tablas, "Alumno" y "Curso". Cada alumno puede matricularse en varios cursos, y cada curso puede tener varios alumnos matriculados. Para representar esta relación de muchos a muchos, se crea una tabla intermedia, digamos "Matrícula". Esta tabla contiene registros que vinculan a estudiantes específicos con cursos específicos, indicando su inscripción.

Las relaciones de muchos a muchos se implementan utilizando claves primarias y claves externas para establecer asociaciones entre las tablas a través de la tabla intermedia.

Comprender y definir correctamente las relaciones entre entidades es esencial para diseñar una base de datos relacional eficaz, ya que garantiza la integridad de los datos y permite realizar consultas y recuperar información relacionada con precisión.

Entidades fuertes y débiles en el Modelo Relacional 🚦

En el modelo relacional, las entidades se representan como tablas, y las relaciones entre entidades se establecen mediante claves primarias y claves foráneas. Las entidades pueden clasificarse como débiles o fuertes en función de sus características y su dependencia de otras entidades.

Definamos ambos tipos de entidades:

1. 💪 Entidad fuerte:

Una entidad fuerte es una entidad que tiene su propio identificador único (clave primaria) y no depende de ninguna otra entidad para su existencia. Puede ser autónoma y tener sus atributos sin necesidad de ninguna relación con otras entidades. Las entidades fuertes tienen atributos que las identifican de forma única.

→ **Ejemplo:** Una tabla "Order" en una base de datos de un e-commerce. Cada pedido puede identificarse de forma única mediante un atributo "OrderID". La tabla "Order" es una entidad fuerte porque tiene su propio identificador único y no depende de ninguna otra tabla para existir.

2. 🏠 Entidad débil:

Una entidad débil es una entidad que depende de otra entidad para su existencia. No tiene su propio identificador único (clave primaria), sino que depende de una relación con una entidad fuerte, junto con su clave parcial (una combinación de sus atributos y la clave primaria de la entidad fuerte relacionada), para formar un identificador único.

→ **Ejemplo:** En una base de datos de una universidad, una tabla "Course" y una tabla "Section". Un "Curso" puede tener múltiples secciones, y cada sección no puede existir sin pertenecer a un curso específico. La entidad "Sección" es una entidad débil porque no tiene un identificador único propio; su identificador se forma combinando su "SectionID" con el "CourseID" (clave parcial) de la tabla "Curso".

Las entidades fuertes tienen su **propio identificador único** y pueden ser independientes, mientras que las entidades débiles **dependen de una relación con una entidad fuerte** y tienen una clave parcial que combina sus atributos con la clave primaria de la entidad fuerte relacionada para formar un identificador único. Comprender la distinción entre entidades fuertes y débiles es crucial para diseñar una base de datos relacional bien estructurada.

Diferencias entre el modelo relacional y el modelo conceptual

El modelo relacional y el modelo conceptual son dos perspectivas diferentes utilizadas en el campo de las bases de datos para representar y diseñar la estructura y organización de los datos.

Veamos las principales diferencias entre ambos modelos:

Modelo relacional:

El modelo relacional es un modelo de datos específico utilizado en los sistemas de gestión de bases de datos relacionales (RDBMS). Es un modelo lógico que representa los datos como una colección de tablas.

Características claves del modelo relacional:

- ★ Los datos se organizan en tablas con filas y columnas.
- ★ Cada tabla tiene un nombre único y contiene registros que representan instancias de entidades.
- ★ Cada columna representa un atributo de la entidad, y cada fila representa un registro único.
- ★ Las relaciones entre tablas se establecen mediante claves primarias y claves externas.

El modelo relacional se centra en proporcionar una forma sencilla y uniforme de organizar y acceder a los datos, por lo que es un modelo muy utilizado en las bases de datos modernas.

Modelo conceptual:

El modelo conceptual, por su parte, es una representación de alto nivel de la estructura de datos y de las relaciones entre las diferentes entidades de una base de datos. Es un modelo no específico de implementación que se utiliza para diseñar y describir la estructura general de la base de datos a nivel conceptual.

Características principales del modelo conceptual:

- ★ Se centra en las relaciones entre las entidades y sus atributos, sin especificar los detalles técnicos de implementación.
- ★ Elimina las complejidades del sistema de gestión de bases de datos subyacente.
- ★ Suele representarse mediante diagramas entidad-relación (ER) o diagramas de clases del Lenguaje Unificado de Modelado (UML).
- ★ Permite a los diseñadores de bases de datos y a las partes interesadas comprender los requisitos y las relaciones de los datos sin entrar en los detalles de tablas y columnas.

El modelo conceptual actúa como puente entre los requisitos de negocio y el diseño lógico en el modelo relacional. Ayuda a los diseñadores de bases de datos y a las partes interesadas a comunicarse y colaborar eficazmente durante las fases iniciales del diseño de la base de datos.

Modelo Conceptual	Modelo Relacional
Propósito: Describe la estructura lógica de los datos sin detalles técnicos.	Propósito: Define cómo se almacenan y gestionan los datos en una base de datos.
Enfoque: Representa entidades, atributos y relaciones de manera abstracta.	Enfoque: Utiliza tablas, columnas y claves para organizar datos.
Nivel de Abstracción: Alto (independiente del sistema de gestión de bases de datos).	Nivel de Abstracción: Bajo (detallado y específico para un SGBD).
Componentes Clave: Entidades, atributos y relaciones.	Componentes Clave: Tablas, columnas, claves primarias y foráneas.
Uso: Ideal para la fase de diseño y análisis de datos.	Uso: Implementación física y manipulación de datos en la base de datos.

En resumen, el modelo relacional es un modelo de datos específico utilizado para la implementación lógica de bases de datos, representado como tablas con filas

y columnas, mientras que el modelo conceptual es una representación de más alto nivel utilizada para diseñar y comprender la estructura general y las relaciones de los datos a nivel conceptual, a menudo representada mediante diagramas entidad-relación. El modelo conceptual sirve de base para crear el modelo relacional.

Reglas de transformación

Las reglas de transformación en bases de datos se refieren a un **conjunto de principios y directrices** que se utilizan para **convertir un modelo conceptual en un modelo relacional**. Estas reglas son esenciales durante el proceso de diseño de una base de datos, ya que permiten representar las entidades, atributos y relaciones en una estructura tabular coherente.

Cuando hablamos de reglas de transformación, generalmente nos referimos a la **transformación de un Modelo Entidad-Relación (ER) a un Modelo Relacional**, que es el tipo más común de modelo utilizado en bases de datos.

Las principales reglas de transformación incluyen:

- ↪ **Entidades:** Cada entidad en el modelo ER se convierte en una tabla en el modelo relacional. Los atributos de la entidad se transforman en columnas de la tabla, y cada registro en la tabla representa una instancia única de la entidad.
- ↪ **Claves primarias:** La clave primaria de una entidad en el modelo ER se convierte en la clave primaria de la tabla relacional correspondiente. La clave primaria garantiza la unicidad de cada registro en la tabla.
- ↪ **Relaciones uno a muchos:** Si una relación entre dos entidades es de tipo "uno a muchos" en el modelo ER, la clave primaria de la entidad "uno" se agrega como clave foránea en la tabla de la entidad "muchos". Esto establece la conexión entre las tablas y refleja la relación entre las entidades.
- ↪ **Relaciones uno a uno:** Las relaciones "uno a uno" se manejan de manera similar a las relaciones "uno a muchos". La diferencia es que, en este caso, la clave primaria de una de las entidades involucradas se convierte en clave foránea en la otra tabla y viceversa.
- ↪ **Relaciones muchos a muchos:** Las relaciones "muchos a muchos" se resuelven mediante la creación de una nueva tabla que actúa como una

tabla de unión. Esta tabla de unión contiene las claves primarias de ambas entidades relacionadas y, si es necesario, otros atributos asociados con la relación.

- ➡ **Atributos compuestos y multivaluados:** Los atributos compuestos se convierten en columnas individuales en la tabla, mientras que los atributos multivaluados se tratan creando una nueva tabla para almacenar esos valores, vinculándola con la tabla principal mediante una clave foránea.

Al seguir estas reglas de transformación, se logra representar adecuadamente la estructura y las relaciones del modelo ER en un modelo relacional, permitiendo implementar una base de datos funcional, normalizada y eficiente.

Normalización y desnormalización de datos

La normalización y desnormalización son dos enfoques opuestos utilizados en el diseño y organización de bases de datos relacionales. Ambos tienen sus ventajas y desventajas, y se aplican según los requisitos específicos del sistema y las necesidades de rendimiento.

Veamos en qué consisten cada uno: 👉

Normalización

La normalización es un proceso mediante el cual se reorganizan las tablas de una base de datos para eliminar redundancias y evitar problemas de actualización, inserción y eliminación anómalos. El objetivo principal de la normalización es garantizar que cada tabla tenga una única fuente de verdad y que los datos se almacenen de manera coherente y eficiente.

La normalización se lleva a cabo siguiendo una serie de reglas o formas normales (1NF, 2NF, 3NF, etc.), cada una de las cuales se basa en ciertos principios y criterios para estructurar los datos de manera óptima. Cuanto mayor es la forma normal a la que se llega, menos redundancia hay en las tablas, lo que puede facilitar el mantenimiento y la comprensión de la base de datos.

A continuación, veamos cuáles son las formas normales más comunes:

1. Primera Forma Normal (1FN):

En esta forma normal, se requiere que cada celda de una tabla contenga un único valor atómico, es decir, indivisible. Además, cada tabla debe tener

una clave primaria única que identifique de manera única cada fila. La 1FN elimina la presencia de atributos repetidos y multivaluados, permitiendo que cada atributo tenga un solo valor.

Ejemplo de 1NF:

Supongamos que tienes una tabla de estudiantes con una columna para los cursos que están tomando:

Estudiante	Cursos
Estudiante A	Matemáticas, Física, Química
Estudiante B	Historia, Inglés

Para llevarlo a 1NF, separa los cursos en registros separados:

Estudiante	Cursos
Estudiante A	Matemáticas
Estudiante A	Física
Estudiante A	Química
Estudiante B	Historia
Estudiante B	Inglés

2. Segunda Forma Normal (2FN):

Para cumplir con la 2FN, una tabla debe estar en 1FN y, además, no puede haber dependencias parciales en la clave primaria. Esto significa que los atributos no clave deben depender completamente de toda la clave primaria, y no solo de una parte de ella. Si existen dependencias parciales, es necesario dividir la tabla en dos o más tablas relacionadas.

Ejemplo de 2NF:

Supongamos que tienes una tabla de pedidos con la siguiente estructura

N Pedido	Producto	Cantidad
Pedido 1	Producto 1	5
Pedido 2	Producto 2	5
Pedido 3	Producto 3	2
<u>Pedido 4</u>	Producto 4	8
Pedido 5	Producto 5	1

En este caso, la clave primaria sería "Número de Pedido" y "Producto" porque juntos identifican de manera única cada fila.

En resumen:

Para aplicar 2NF, crea dos tablas separadas: una para pedidos y otra para detalles de pedido.

Clave Pedido	Nro Pedido
1	Pedido 1
2	Pedido 2
3	Pedido 3
4	<u>Pedido 4</u>
5	Pedido 5

Clave Pedido	Producto	Cantidad
1	Producto 1	5
2	Producto 2	5
3	Producto 3	2
4	Producto 4	8
5	Producto 5	1

3. Tercera Forma Normal (3FN):

La 3FN se basa en los principios de la 2FN y establece que no debe haber dependencias transitivas en la tabla. Es decir, los atributos no clave no deben depender funcionalmente de otros atributos no clave. Si hay dependencias transitivas, se deben separar las tablas para evitar la redundancia de datos y mantener la integridad.

Ejemplo de 3NF:

Supongamos que tienes una tabla de empleados con información sobre sus departamentos y ubicaciones:

Empleado	Departamento	Ubicación
Empleado 1	Ventas	Ciudad A
Empleado 2	Marketing	Ciudad B
Empleado 3	Ventas	Ciudad C

En este caso, la ubicación está relacionada con el departamento, no directamente con el empleado.

En resumen:

Para aplicar 3NF, crea una tabla separada para los departamentos y sus ubicaciones correspondientes.

Empleado	Departamento
Empleado 1	Ventas
Empleado 2	Marketing
Empleado 3	Ventas

Departamento	Ubicacion
Ventas	Ciudad A
Marketing	Ciudad B
Ventas	Ciudad C

4. Cuarta Forma Normal (4FN):

La 4FN se aplica a tablas que ya cumplen con la 3FN. Esta forma normal aborda el problema de las dependencias multivaluadas, lo que significa que un atributo puede depender de varios valores de otro atributo. Para cumplir con la 4FN, se deben crear nuevas tablas para eliminar estas dependencias multivaluadas y evitar la redundancia de datos.

5. Quinta Forma Normal (5FN):

También conocida como Proyección unión o Forma Normal de Proyección, la 5FN se refiere a la eliminación de dependencias de unión en una tabla. Esto ocurre cuando una tabla se puede dividir en dos o más tablas independientes que se pueden unir nuevamente mediante una relación en común. La 5FN se aplica en casos muy específicos y no siempre es necesario alcanzarla en todos los diseños de bases de datos.

Cabe mencionar que existen formas normales adicionales, como la Sexta Forma Normal (6FN) y más allá, pero su aplicación es mucho menos común y se relaciona con situaciones más complejas que generalmente no se encuentran en bases de datos típicas.

Desnormalización

La desnormalización, por otro lado, es el proceso de volver a introducir redundancia en una base de datos normalizada para mejorar el rendimiento en consultas y operaciones. Esto implica combinar tablas o agregar duplicados selectivos de datos para evitar un alto número de joins y reducir el tiempo de respuesta en ciertas consultas.

La desnormalización es útil en situaciones en las que el rendimiento de las consultas es crítico y se necesita una alta velocidad de recuperación de datos. Sin embargo, este enfoque puede aumentar la complejidad del mantenimiento

de la base de datos y aumentar la posibilidad de inconsistencias de datos si no se maneja adecuadamente.

La decisión de aplicar normalización o desnormalización en el diseño de una base de datos dependerá de los requisitos específicos del sistema, la complejidad del modelo de datos y las necesidades de rendimiento. En algunos casos, una combinación de ambos enfoques puede ser la solución más adecuada para lograr un equilibrio entre la eficiencia y la coherencia de los datos.

Diccionario de Datos

Es un componente fundamental en las bases de datos que actúa como un **repositorio centralizado** donde se almacena información detallada sobre los datos de la base de datos. Esta información incluye la **estructura, características y relaciones de todos los objetos** que forman parte de la base de datos, como **tablas, columnas, índices, vistas, restricciones** y más.

En esencia, el diccionario de datos documenta cómo están organizados los datos y cómo se utilizan. Esto incluye:

- ⇒ Nombres y descripciones de tablas y columnas
- ⇒ Tipos de datos (por ejemplo, INT, VARCHAR, DATE)
- ⇒ Restricciones (como claves primarias, foráneas, restricciones de unicidad)
- ⇒ Relaciones entre tablas
- ⇒ Índices utilizados para optimizar las consultas
- ⇒ Usuarios y permisos asociados a los objetos de la base de datos

Este diccionario sirve para la documentación interna, pero también es utilizado por el Sistema de Gestión de Bases de Datos (SGBD) para gestionar y validar las operaciones que se realizan sobre la base de datos. Por ejemplo, al crear una tabla o ejecutar una consulta, el SGBD consulta el diccionario de datos para verificar que las columnas y relaciones sean correctas.

¿Cuál es su impacto?

- ★ **Facilita el desarrollo y mantenimiento:** Ayuda a los desarrolladores a comprender las relaciones y restricciones sin tener que revisar el código o el esquema directamente.
- ★ **Mejora la precisión en las consultas:** Conocer las relaciones y restricciones reduce errores en las consultas SQL.
- ★ **Asegura la integridad de los datos:** Las restricciones garantizan que los datos sean consistentes y válidos.

Aplicación del Diccionario de Datos 📝

Imagina una base de datos para una aplicación de wallet digital. El Diccionario de Datos podría detallar la siguiente tabla:

Tabla: Transacciones

- **Descripción:** Almacena el registro de todas las transacciones realizadas en la wallet.
- **Columnas:**
 - ↪ **TransaccionID:** INT, Clave primaria, Identificador único para cada transacción.
 - ↪ **Fecha:** DATETIME, Fecha y hora en que se realizó la transacción.
 - ↪ **Monto:** DECIMAL, Cantidad de dinero transferida (positivo para ingresos, negativo para egresos).
 - ↪ **Tipo:** ENUM ('Ingreso', 'Egreso'), Especifica si es una carga de saldo o un pago.
 - ↪ **UsuarioID:** INT, Clave foránea que relaciona la transacción con el usuario que la realizó.
 - ↪ **MetodoPagoID:** INT, Clave foránea que indica el método de pago (tarjeta, cuenta bancaria, etc.).
 - ↪ **Estado:** ENUM ('Pendiente', 'Completada', 'Fallida'), Estado actual de la transacción.

Relaciones:

- UsuarioID está relacionado con UsuarioID en la tabla Usuarios, lo que permite saber quién realizó la transacción.
- MetodoPagoID está relacionado con MetodoPagoID en la tabla MetodosPago, que detalla las opciones de pago disponibles.

Restricciones:

- TransaccionID es único y no puede ser nulo.
- Monto debe ser un valor distinto de cero.
- Estado solo puede tener uno de los valores definidos en el ENUM.

Preservar un uso adecuado del **Diccionario de Datos** facilita el desarrollo y mantenimiento de los datos **garantizando la integridad y consistencia** de los mismos a lo largo del tiempo.

Cierre

Durante esta lección hemos visto los fundamentos del Modelo Entidad-Relación en bases de datos, los distintos tipos de relación entre entidades, así como los tipos de datos y restricciones aplicables al modelo. También hemos aprendido sobre algunas reglas de transformación para convertir un modelo Entidad-Relación en uno relacional y las técnicas de normalización para garantizar la eficiencia y consistencia de la base de datos. Aplicar estos conceptos te permitirá diseñar y gestionar bases de datos más consistentes y bien estructuradas.

Referencias

- **Beaulieu, A.** (2009). *Learning SQL* (2nd ed.). O'Reilly Media. https://www.r-5.org/files/books/computers/languages/sql/mysql/Alan_Beaullieu-Learning_SQL-EN.pdf
- **Groff, J. R., Weinberg, P. N., & Oppe, A.** (2011). *SQL: The complete reference* (3rd ed.). McGraw-Hill.
- **Microsoft.** (n.d.). *SQL Server documentation*. <https://docs.microsoft.com/en-us/sql/>
- **Mode Analytics.** (n.d.). *SQL tutorial*. <https://mode.com/sql-tutorial/>
- **SQLCourse.com.** (n.d.). *SQL tutorial – Beginner & advancer*. <https://www.sqlcourse.com/>

¡Muchas gracias!

Nos vemos en la próxima lección

